NAVIGATION / URL TRUST / IDENTITY FLOWS

Open Redirect Explained

Understand URL trust, prove unsafe navigation, and protect identity flows

PROMISED JOURNEY

A trusted link starts the trip.

ACTUAL DESTINATION

Policy must control the exit.

Understand the journey before the vulnerability.
Find the input. Follow the hops. Verify the repair.
Practice with local receivers and dummy identity flows.

Haroon Shahid

First edition / Version 1.0 / 1 November 2026

Before you begin

Open Redirect Explained

Understand URL trust, prove unsafe navigation, and protect identity flows

By Haroon Shahid. First edition, version 1.0. 1 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework, identity-provider, and protocol guidance is version-sensitive. The local lab uses fixed loopback origins, dummy markers, and a simplified identity model. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 48 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Wayfinder lab.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. A trusted address is a starting point 7

02. The web request behind the journey 9

03. Read a URL one piece at a time 11

Sources and further reading 13

Introduction

Follow the journey, not a list of strings

A link begins at a familiar address. The application reads a destination, the browser follows an instruction, and a different page opens. That observation can be a real security finding. It is not automatically token theft, script execution, or account takeover.

This book teaches the whole reasoning path: what the user expects, what the browser processes, what the application permits, and what trust changes at each step. The aim is to help you predict and verify behavior rather than memorize a collection of unusual URLs.

You can begin without knowing HTTP or OAuth. Chapters 1-8 explain web messages, URL components, origins, navigation, sessions, and token purposes. The next stages build controlled discovery and proof, data-flow analysis, identity understanding, defenses, and professional review.

What you will learn to do

You will identify destination sources and navigation sinks; read the effective authority instead of a familiar substring; preserve first-hop and browser evidence; analyze queries, fragments, referrers, methods, and delayed returns; distinguish code exposure from successful redemption; and build a context-specific repair with allow/deny tests.

The strongest identity lesson is separation. Registered callback, transaction binding, PKCE, identity assertion validation, and post-login continuation are related controls with different responsibilities. A correct control at one layer does not approve every other layer.

Practice with a visible model

Wayfinder is the bundled local lab. Three fixed loopback origins represent an application, receiver, and simplified identity server. You can inspect real HTTP responses, navigate in a browser, observe dummy request data, compare safe and weak policies, and reset the model.

The lab uses no real credentials or external targets. Its selected code-flow checks are not a full OAuth/OIDC implementation. Framework, provider, proxy, cache, and distributed-system guidance requires deployment-specific verification. The validation report states what was actually executed.

How to study

Predict before revealing answers. After a mission, record source, interpretation, policy, sink, browser journey, data, and supported consequence. Use the revision atlas for recall and the flow libraries for architecture. When a term interrupts understanding, use the linked glossary and return to the example.

A book cannot certify expertise or honestly promise that no other resource can match it. The useful standard is whether you can explain a new authorized journey, test its assumptions, repair a failed contract, and state the remaining uncertainty. The mastery roadmap measures those artifacts.

Practice only on systems and accounts you own or are explicitly authorized to assess. A minimal controlled marker is often stronger evidence than an uncontrolled collection of real data. Careful proof is part of advanced skill.

Learning roadmap

Reader	Route	Evidence to produce
New learner	Read in order from Chapter 1	URL map, predictions, first-hop/arrival proof
Developer	Foundations, then 33-42	Destination contracts, sink coverage, regression tests
Authorized tester	Foundations, 9-32, then 41-47	Controlled journey, precondition table, bounded report
Architect	Foundations, identity, defenses, capstone	Consumer map, protocol/continuation separation, acceptance plan

Six stages and evidence gates

1. **Foundations, 1-8.** Understand URL meaning, navigation, origins, methods, sessions, and credentials. Gate: explain why familiar text, HTTPS, and authentication do not approve every destination.

2. **Discovery and proof, 9-16.** Inventory journeys and sources. Prove instructions and arrival. Understand parser and chain errors. Gate: a reproducible controlled navigation proof.

3. **Consequences, 17-24.** Follow data, referrers, POST bodies, tokens, proxies, and caches. Gate: an impact claim with each required condition labeled proven, blocked, or untested.

4. **Identity, 25-32.** Map OAuth/OIDC/SSO roles, exact callbacks, state, nonce, PKCE, code leakage, and legacy fragments. Gate: explain why one exposed value may still be unusable.

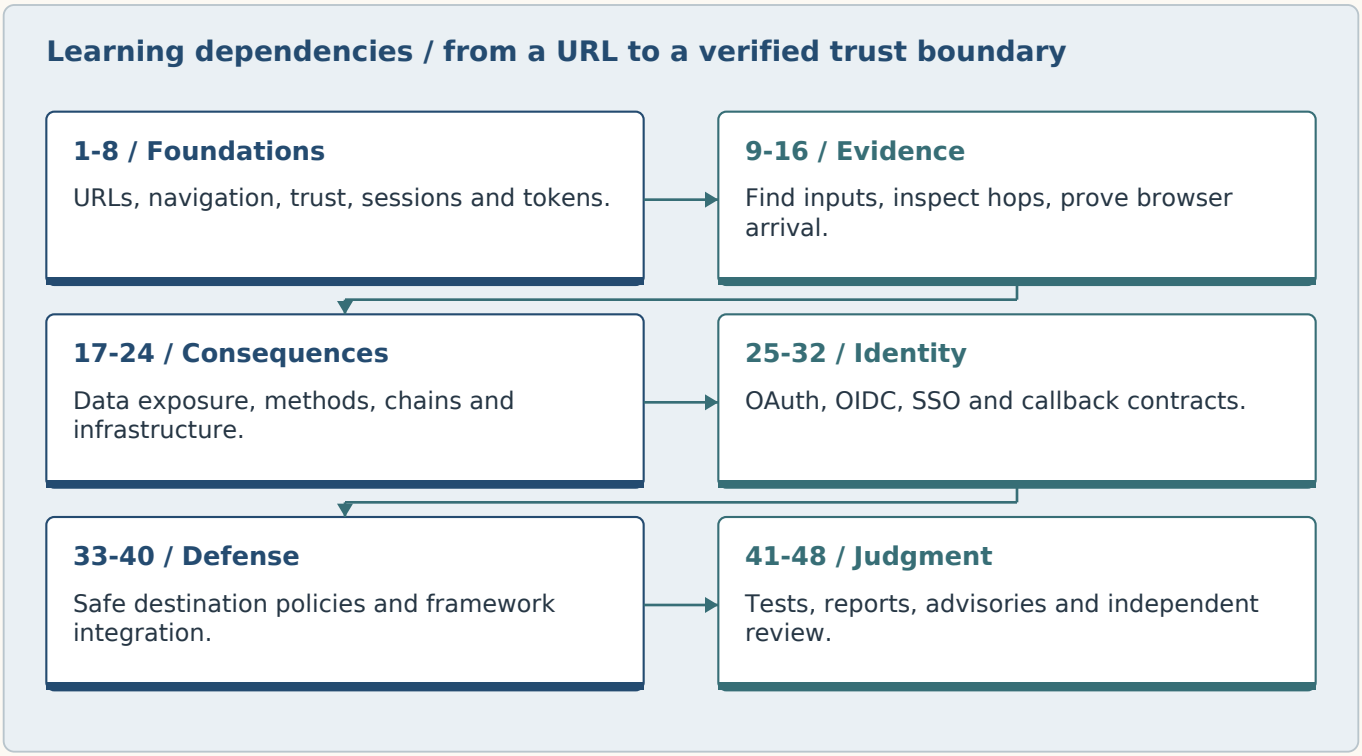
5. **Defense, 33-40.** Choose narrow contracts and implement them through framework and provider paths. Gate: a repair preserving legitimate journeys while denying unintended targets.

6. **Professional judgment, 41-48.** Verify consumers, read advisories, report, remediate, solve the capstone, and transfer the method. Gate: independently review a new authorized journey with honest coverage limits.

Use the companion system

The prediction deck trains reasoning before sending. The 32 missions connect concepts to local effects. The 24-case clinic trains classification and partial-fix judgment. The notebook, hop ledger, claim table, report, and regression templates preserve evidence. The redirect, authentication, and OAuth flow libraries support review.

Use flexible study sessions rather than a promise of mastery in a fixed number of hours. Read one connected group, predict two outcomes, perform one exercise, and write one rule in plain language. Revisit it after a day, a week, and a month.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

I. UNDERSTAND NAVIGATION AND TRUST

01. A trusted address is a starting point

Learn to: Separate the link a person trusts from the destination that actually opens.

Think of it this way: A shop can have a trusted front door and still give visitors directions to the wrong building.

01A / Four different views of one link

User expectation

The familiar starting domain seems reassuring.

Browser instruction

A response or script selects another URL.

Application promise

Only intended destinations should be chosen.

Trust failure

Caller input selects an unintended exit.

A redirect becomes a security issue when the actual journey violates the intended destination policy.

Begin with a journey

A web link names a place to visit. A person may trust the name because it belongs to their bank, workplace, or favorite store. After opening it, the browser may receive an instruction to visit another address. That second address is a separate security decision.

A **redirect** changes the destination of a request or navigation. Redirects are normal. A site may move an old article, send a signed-in user to their dashboard, or return a customer from a payment provider. The security question is who controls the destination and which destinations the application intends to allow.

An **open redirect** exists when untrusted input can make an application direct a user to a destination outside its intended policy. The problem can be in a server response, a script, a stored link, or a chain of components. A parameter named `next` is a candidate, not proof by itself.

Trust has a scope

Trust means relying on something for a particular purpose. Trusting a store to display an order does not mean trusting every website to which it can send you. A familiar starting domain does not make the final domain part of that organization.

Browsers are not people. They follow parsing, navigation, and security rules. They do not decide that a destination is honest because a company used it. Throughout this book, "what the browser sees" means the URL and instructions it processes, not human judgment.

A small example

```
https://shop.example.test/continue?next=DESTINATION
```

The site may intend `DESTINATION` to be a local order page. If a caller can instead select an unrelated receiver and the browser arrives there, the navigation policy may have failed. If the site deliberately offers a clear external-link service, the same observation can be intended behavior. Establish the policy before the conclusion.

The `.test` names in the book are fictional. The executable lab uses fixed loopback addresses on your own computer. It never asks you to collect real passwords or test an unrelated website.

Three promises to keep separate

Promise	What it does not promise
The starting address is genuine	Every later destination is genuine
The connection uses HTTPS	The destination is approved or honest
The redirect is confirmed	A token or account was compromised

Practice: describe a login journey you use. Name the starting page, identity provider if any, callback, and final page. It is fine not to know all four yet. The book will teach the missing terms before using them in advanced examples.

Remember: follow the whole journey. A familiar first address is evidence about the first address, not a guarantee about the last one.

Key takeaway. A trusted starting URL does not authorize every destination. Prove the journey and its violated policy.

Check your understanding

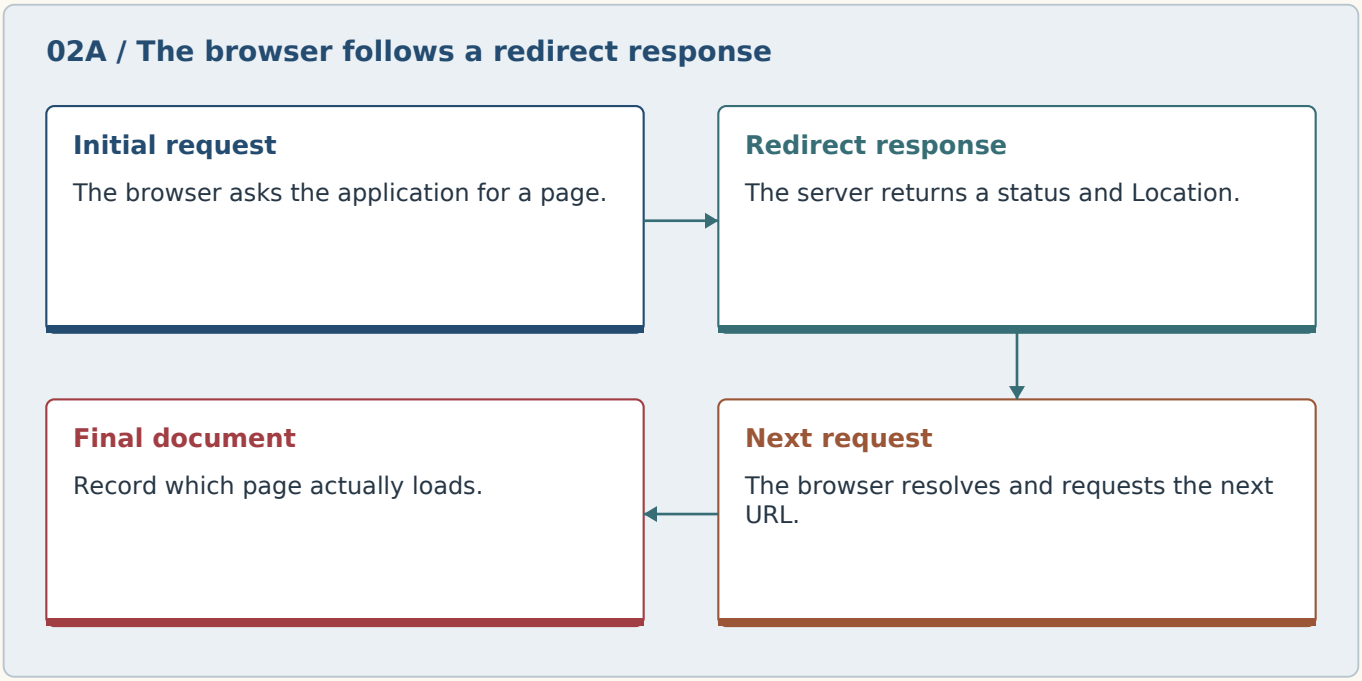
1.1 Does a next parameter prove a vulnerability?	**1.2** Does HTTPS make a destination approved?	**1.3** Does every confirmed redirect steal a token?

Answer explanations are in the full ebook. Sources: OWASP, CWE

02. The web request behind the journey

Learn to: Understand browser, server, request, response, and navigation.

Think of it this way: A visitor asks a reception desk for a room; the desk can answer or send the visitor to another address.



The redirect response and the browser arrival are related but distinct observations.

The pieces of a web application

A **browser** is software that displays web pages and makes requests. A **server** receives requests and returns responses. An application is the logic that handles those messages. It may use databases and other services behind the scenes.

The internet connects networks. DNS helps resolve a hostname to addressing information. HTTP defines web messages. HTTPS adds TLS protection to the connection. These mechanisms help reach a service; the application still needs to decide where a redirect may lead.

A request includes a method, target, headers, and sometimes a body. A response includes a status, headers, and sometimes a body. Headers carry metadata. A body carries content such as HTML, JSON, or submitted form values.

```
GET /continue?next=%2Faccount HTTP/1.1
Host: shop.example.test
```

The response could be:

```
HTTP/1.1 302 Found
Location: /account
```

`Location` supplies the next URL reference for this redirect response. The browser resolves it, makes the next request, and may repeat the process. The server does not physically move the browser or necessarily fetch the next site itself.

Navigation is not every request

A **navigation** loads a document into a browsing context, such as a tab or frame. A page also fetches images, scripts, styles, and API data. A script's `fetch()` request can follow redirects without changing the tab's address. Seeing a redirected API request does not by itself prove that a user is sent to another page.

A server-side HTTP client can also follow a redirect. That is a different consumer with different consequences. If it contacts an internal service, the relevant issue may be SSRF. Later chapters keep browser navigation and server fetching separate.

Observe rather than assume

Use your browser's Network panel on your own lab. Keep the request log during navigation. Record the initial request, each response status and Location, and the final document URL. A replay tool can inspect a first response without automatically following it, which helps preserve evidence of the original sink.

A **sink** is the operation that consumes a value in a security-relevant way. For this book, a redirect response or navigation API can be a sink. A **source** is where the potentially untrusted destination comes from.

Practice: draw a browser, application, and receiver. Mark who sends each request. If the browser contacts the receiver, it is a browser journey. If the application contacts it, explain that separate server-side behavior.

Misconception: "The server returned a URL, so the user was redirected." A JSON string or ordinary link may not execute navigation. Establish the consumer and actual effect.

Key takeaway. Record who follows the redirect. A browser navigation, API fetch, and server-side fetch are different effects.

Check your understanding

2.1 Does fetch always change the tab URL?	**2.2** Who normally contacts the next site after an HTTP navigation redirect?	**2.3** What is a sink?

Answer explanations are in the full ebook. Sources: HTTP, FETCH, NAV

03. Read a URL one piece at a time

Learn to: Identify scheme, authority, host, port, path, query, and fragment.

Think of it this way: An address has a country, building, room, and delivery note. A name in the note is not the building address.

03A / Read the destination by component

`https://login.example.test:443/account?next=home#step`

Scheme + authority	https / login.example.test / port 443
Path	/account selects a route at that authority.
Query	next=home is data sent in the request.
Fragment	#step is normally handled by the browser.

The host controls the network destination; names appearing elsewhere are data.

The visible string has structure

A URL is a structured address. Consider:

```
https://login.example.test:443/account?next=home#step
```

The **scheme** is `https`. The **host** is `login.example.test`. The **port** is 443 here; a default port may disappear when a URL is serialized. The **path** is `/account`. The **query** is `next=home`. The **fragment** is `step` after #.

The **authority** is the part that identifies the network authority, potentially including user information, host, and port. Do not find it by searching for a trusted name anywhere in the string.

Query data is not the host

In this URL, the host is `receiver.example.test`:

```
https://receiver.example.test/?label=shop.example.test
```

The familiar shop name is query data. Similarly, a path such as `/shop.example.test/orders` does not make that name the host. A browser's parsed components provide more reliable evidence than visual scanning.

A query can contain named parameters separated by &. Their format and duplicate-value behavior depend on the application. A redirect parameter often contains a nested URL. That inner value has its own components after the outer query parser interprets it.

Fragments are different

The fragment is normally not sent as part of the HTTP request target. A page script can read it after navigation. A redirect can also affect whether an existing fragment is retained by the browser. This distinction is essential for understanding legacy token-in-fragment flows later.

Do not conclude that "not sent in HTTP" means "cannot leak." An arriving page can observe its own fragment through script. Conversely, a receiver’s raw server log alone cannot prove what fragment the browser held.

Structured parsing

In a browser or modern Node.js:

```
const u = new URL('https://login.example.test/account');
console.log(u.protocol, u.hostname, u.port, u.pathname);
```

This demonstrates parsing, not a complete validator. You still need a destination policy, a trusted base for relative references, and agreement with the final consumer.

Practice

For three fictional URLs, mark the host without following them. Put one trusted-looking name in a path, one in a query, and one in user information. Compare the parser’s hostname result. Use only the lab’s parse-only bench for unusual strings; it does not navigate or make a network request.

Common mistake: treating a trusted word anywhere in a URL as proof of a trusted destination. Authority comes from the parsed URL structure, not the most recognizable substring.

Key takeaway. Identify the parsed host and effective URL. Query, path, and fragment text do not change who owns the authority.

Check your understanding

3.1 Is shop.example.test in a query the destination host?	**3.2** Is a fragment sent in the ordinary request target?	**3.3** Does new URL validate business permission?

Answer explanations are in the full ebook. Sources: URL, URI, MDNURL

Sources and further reading

Research cutoff: 1 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

OWASP - OWASP: Unvalidated Redirects and Forwards

Consulted. Root weakness and defensive design. The Wayfinder examples and diagrams are original.

https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

CWE - MITRE CWE-601: URL Redirection to Untrusted Site

Targeted further reading. Classification and consequences; classification alone does not prove an application finding.

https://cwe.mitre.org/data/definitions/601.html

URL - WHATWG URL Living Standard

Consulted. Components, relative resolution, special-scheme parsing, userinfo and serialization. Check actual consumers too.

https://url.spec.whatwg.org/

HTTP - RFC 9110: HTTP Semantics

Consulted targeted sections. Redirection, methods, Location and fragment inheritance. Client behavior requires runtime verification.

https://www.rfc-editor.org/rfc/rfc9110.html

URI - RFC 3986: URI Generic Syntax

Targeted further reading. URI components and resolution; browser URL processing is specified separately.

https://www.rfc-editor.org/rfc/rfc3986.html

NAV - WHATWG HTML: Navigation and session history

Targeted further reading. Navigation, browsing contexts and history.

https://html.spec.whatwg.org/multipage/nav-history-apis.html

FETCH - WHATWG Fetch Standard

Targeted further reading. Redirect modes, requests, responses and CORS. A fetch result is not a top-level navigation proof.

https://fetch.spec.whatwg.org/

MDNURL - MDN: URL API

Targeted further reading. Structured URL parsing in browsers.

https://developer.mozilla.org/en-US/docs/Web/API/URL